hhu

hhu
Heinrich Heine
Universität Düsseldorf

HeineOS

Assignment Material: Assembly

Prof. Dr. Michael Schöttner
Dr. Fabian Ruhland

What is an Assembler?

Heinrich Heine Universität Düsseldorf

- A simple compiler, that translates assembly code into machine language
  - Assembly code = human-readable CPU instructions
  - Machine language = Binary representation of CPU instructions
- Assembly language is more comfortable than machine language
  - Instead of `000001011110100000000011` the programmer can write `add ax,1000`
- Assembly instructions are uniquely mapped to binary machine instructions

Assembly language	**Machine language**
`add ax`	`00000101`
`1000` (decimal)	`0000001111101000`

- Every CPU architecture has its own assembler

What is an Assembler?

Heinrich Heine Universität Düsseldorf

- Contrary to real (higher level language) compilers, an assembler cannot translate many complex expressions
- Assembly language
  - Assembly code = human-readable CPU instructions
- Higher level constructs are broken down into simpler instruction by a compiler
  - No complex instructions
  - No convenient loops (mostly just *goto* equivalents)
  - No structured datatypes
  - No subroutines with parameter passing

Example

- The expressions `sum = a + b + c + d` is too complex for an assembler and needs to be split into multiple instructions
- The x86 assembler can only add two numbers at a time and store the result in one of the used *variables* (**accumulator register**)
- The above expressions corresponds to these instruction in an assembly program:

```
sum = a;
sum = sum + b;
sum = sum + c;
sum = sum + d;
```

Example

- This program:

```
sum = a;
sum = sum + b;
sum = sum + c;
sum = sum + d;
```

  would look like this in x86 assembly language:

```
mov rax, [a]
add rax, [b]
add rax, [c]
add rax, [d]
```

- Assemblers only support primitive operations
- Most assembler work line-oriented
  - One line matches one machine instruction
  - No semicolon necessary at the end of a line

Branch instruction *if*

Heinrich Heine Universität Düsseldorf

- Simple if-then-else constructs are too complex for an assembler

```
if (a== 4711) {

    ...
} else {
    ...
}
```

- In x86 assembly language, this would look like the following:

```
    cmp rax, 4711   ; compare eax to 4711
    jne unequal     ; if rax != 4711 → jump to unequal

equal: ...          ; otherwise, continue here (if-branch)
    jmp after       ; jump over else-branch

unequal: ...        ; else-branch

after: ...          ; continue with the rest of the program
```

A simple *for* loop

Heinrich Heine Universität Düsseldorf

- Simple counting loops are supported by the x86 instruction set
- The following loop...

```
for (i = 0; i < 100; i++) {
    sum = sum + a;
}
```

- ...looks like this in assembly language:

```
    mov rcx, 100
loop_start:
    add rax, [a]
    loop loop_start
```

- The `loop` instruction implicitly decrements the `rcx` register and only jumps to the specified label, if `rcx` is not 0.

CPU Register

Heinrich Heine Universität Düsseldorf

- Assembly language does not support arbitrary variables → Parameter values or computation results are stored in CPU registers
- A CPU register is an extremely fast, but very small, temporary storage inside the CPU, that can store 64 bits in x86_64 CPUs (extensions like SSE or AVX introduce larger registers with up to 512 bits for vector operations)
- The actual variables of a high level language are translated to memory addresses inside the data segment of an object file
- To perform calculations using a variable, its value must first be loaded into a CPU register from the corresponding memory address
  - Only a limited amount of variables fit into the few available registers
  - The mapping `variable <=> register` changes over time

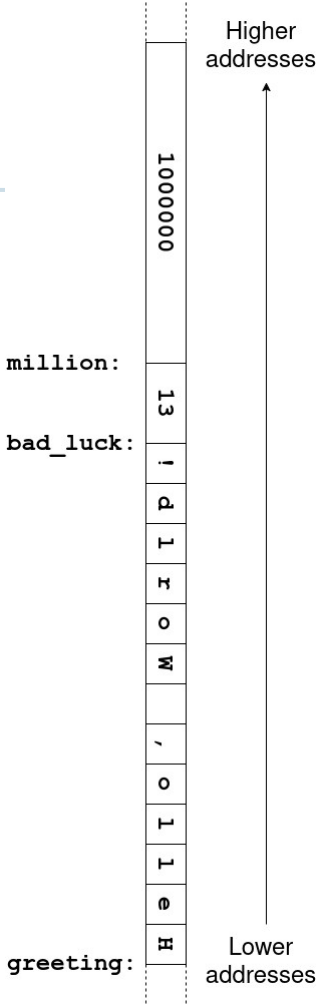
Memory

- Oftentimes, a problem cannot be solved using register alone
  - Accessing main memory is necessary
- The main memory acts like a giant array of registers, that may be 8, 16, 32 or 64 bit wide
  - A byte is the smallest addressable unit
  - Memory cells are enumerated (index)
  - Access times are much slower (~70 ns) compare to CPU registers (~1 ns)
- To access a value in main memory, the programmer must know its index inside the memory → This is called the value's *address*
- Main memory indexed in bytes, ascending from 0

Memory: Example

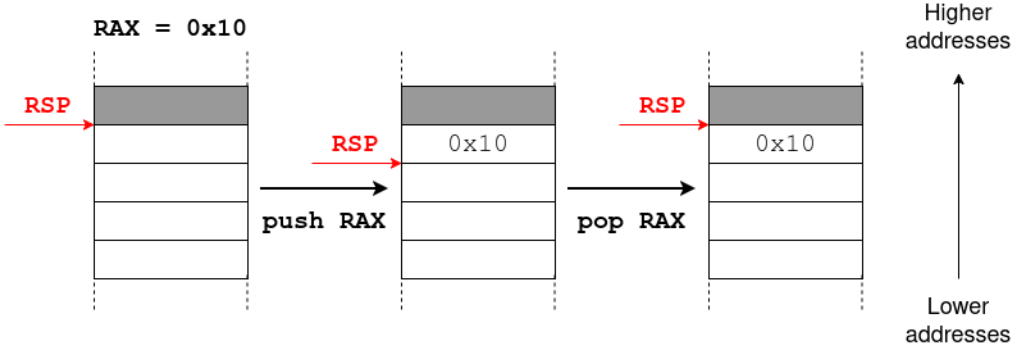
```
[SECTION .data]
greeting:    db 'Hello, World!'
bad_luck:    dw 13
million:     dq 1000000

[SECTION .text]
mov rax, [million]
```

db: define byte
dw: define word (2 byte)
dd: define double word (4 byte)
dq: define quad word (8 byte)



Stack

- The `push` instruction puts values *on top* of the stack, while `pop` removes the topmost value from the stack
  - The current memory address for `push`/`pop` is stored in a special registers called *Stack Pointer* (`RSP` register in x86_64) → Always points to the last used entry
  - Programmers do not need to worry about the concrete stack pointer value, only about the order in which values were pushed onto the stack
  - `push` and `pop` always use 32 or 64 bit values (depending on the CPU mode)



Addressing modes

Heinrich Heine Universität Düsseldorf

- Most x86 instructions can take operands either out ouf registers, the main memory or immediate constants
- The `mov` instruction works with the following addressing modes, where the first operand ist always the target and the second operand is always the source (**NASM-syntax**)
  - **Register addressing:** The value of one register is transferred into another one
    ```
    mov rbx, rdi
    ```
  - **Immediate addressing:** A constant value is loaded into a register
    ```
    mov rbx, 1000
    ```
  - **Absolute addressing:** The value stored at a given memory address is transferred into a register
    ```
    mov rbx, [1000]
    ```
  - **Register indirect addressing:** The value stored at the memory address, pointed to by the second register, is loaded into the first register
    ```
    mov rbx, [rax]
    ```
  - **Base register addressing:** The value stored at the memory address, pointed to by the sum of the second register's content and a constant value, is loaded into the first register
    ```
    mov rax, [10+rsi]
    ```

Functions

- Higher level languages introduce the concept of *functions* or *procedures*
  - In contrast to a simple `goto`, procedures may be called from anywhere in the program and after it returns, the program continues automatically with next instruction after the procedure call
  - The procedure does not need to know from where it was called where the program continues afterward, as this automatically handled using the stack
- Not only its data, but the program itself (code) resides in main memory
  → Every machine language instruction is uniquely addressable in memory
- To let the processor execute a program, its instruction pointer needs to point to the address of the program's first machine language instruction
  - The instruction pointer itself is a special register (`rip` for x86_64 CPUs)
  - The processor executes the instruction pointet to by `rip` and then automatically increases `rip` by the length of the executed instruction

Functions

- A jump instruction (e.g. `jmp`, `jne`, `je`, …) does not increase the `rip` by its length, but …
  - … either sets the `rip` directly to the given address (absolute jump)
  - … or increases/decreases the `rip` by a relative offset
- Calling a function/procedure behaves similar to a jump instruction, but also pushes the old `rip` value (+ instruction length) beforehand (this is called the *return address*)
- When returning from a function, the return address is popped from the stack and the CPU jumps to it.
- In the x86 instruction set, `call` implicitly stores the return address on the stack and jumps to a function, while `ret` returns from a function to the return address on the stack:

```
; ------- Main program -------
main:  ...
       call f1

xyz:   ...

; ------- Function f1 -------
f1:    ...
       ret
```

Application Binary Interface (ABI)

- If a function should take parameters, these are passed in CPU registers or on the stack
- How this works exactly is specified by the **ABI**
  - Size and byte order of primitive data types
  - Calling conventions: Where are parameters and return values stored (Register/Stack)
  - In general, the ABI describes an interface between two binary parts of a program
- We use the *System V ABI for AMD64*
  - https://www.uclibc.org/docs/psABI-x86_64.pdf

Functions with parameters

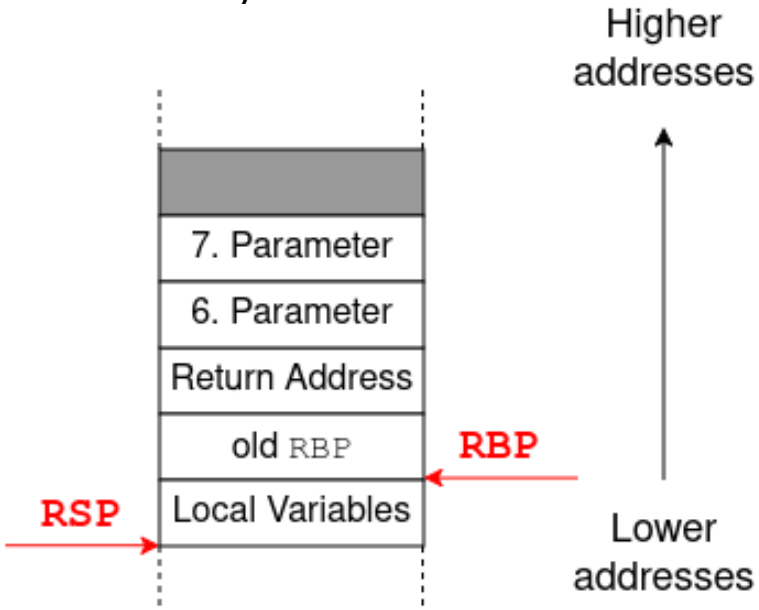
Heinrich Heine Universität Düsseldorf

- If a function takes parameters, the first six parameters are passed in registered, and any additional ones on the stack (x86_64 with System V ABI)
  - Naturally, this has to be prepared before executing the `call` instruction
- Afterward, the parameters need to be cleaned up, either by using multiple `pop` instructions, or by directly updating the stack pointer (`rip`).
  - Needs to be done after the `call` instruction, or when returning from the function (e.g. `ret 16`)

Parameter number	Register/Stack
1	`rdi`
2	`rsi`
3	`rdx`
4	`rcx`
5	`r9`
6	`r9`
>6	stack

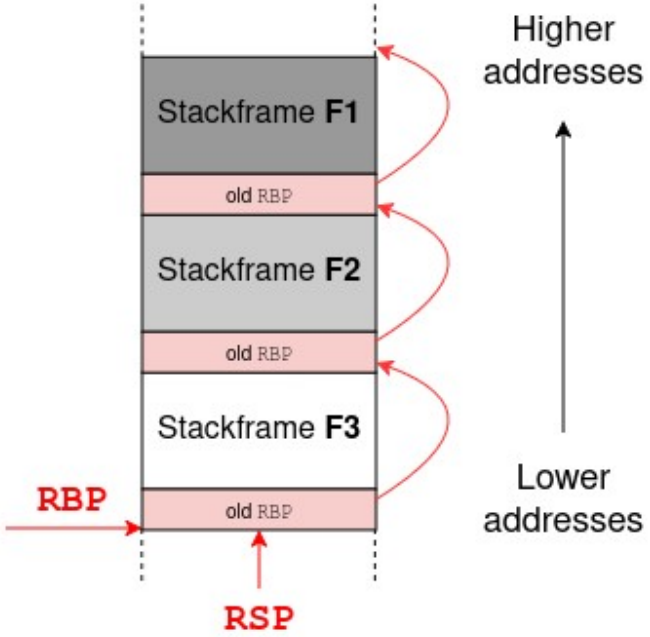
Typical Stackframe

- Addressing of parameters on the stack via `rbp` register with a positive offset
- Addressing of local variables on the stack via `rbp` register with a negative offset
- In 64-bit mode, each stack entry is 64 bit wide



Chaining Function Calls

- Example: Calling order **F1** → **F2** → **F3**
- Stack frames are chained via the `RBP` entries of each frame



Integrating Assembly Code in Rust

Heinrich Heine Universität Düsseldorf

- For our coroutine implementation, we use `naked` functions that may only contain assembly code

```
#[unsafe(naked)]
unsafe extern "C" fn coroutine_start(stack_ptr: usize) {
    unsafe {
        naked_asm!(
            "...", // 1. ASM instruction
            "...", // 2. ASM instruction
            "...", // 3. ASM instruction
            ...
        )
    }
}
```

- The first parameter `stack_ptr` is passed via the `rdi` register
- More informationo on inline assembly in Rust:

https://doc.rust-lang.org/reference/inline-assembly.html